



Working with Data: Retrieval and Vector Stores in LangChain

Document loaders and data ingestion in LangChain



Deepali Srivastava

Author

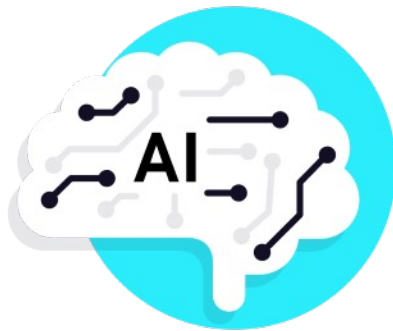
[linkedin.com/in/deepali-srivastava/](https://www.linkedin.com/in/deepali-srivastava/)



Introduction to document loaders and data ingestion



Bringing data into LLM applications



LLM limitation

LLMs cannot directly read files, websites, or databases



External data sources

Real-world applications rely on data from PDFs, CSVs, and web pages

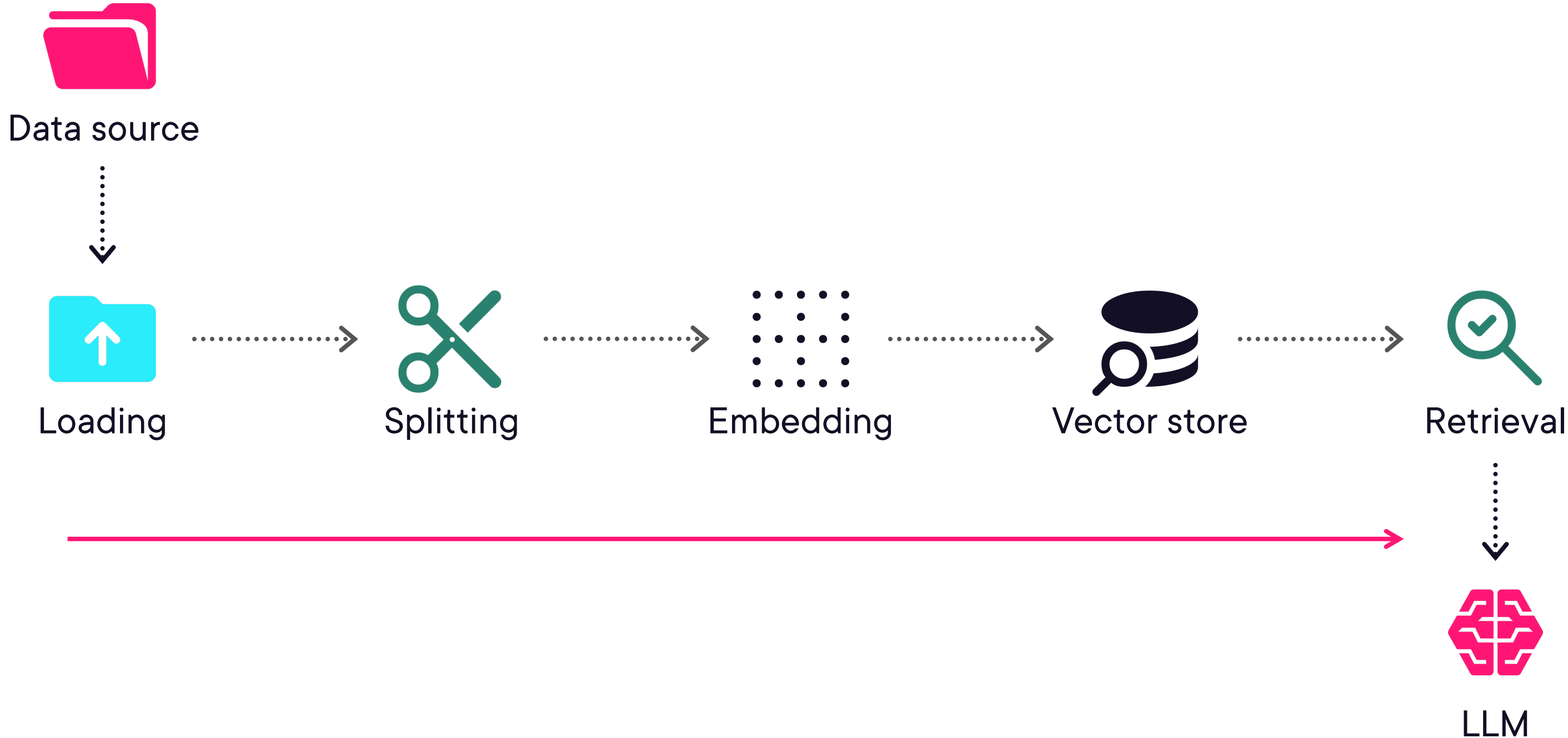


Data ingestion

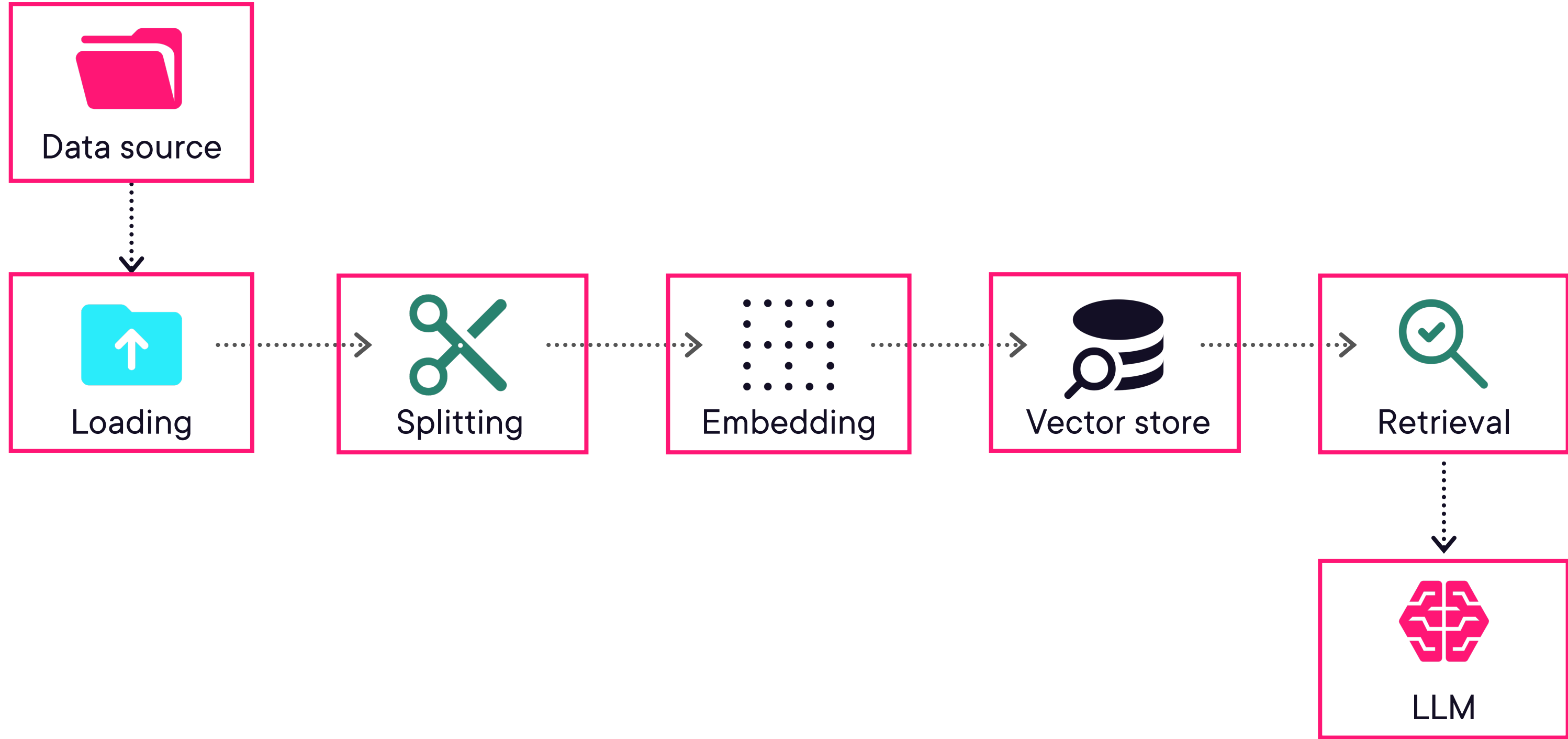
Data must be extracted and converted into text the model can process



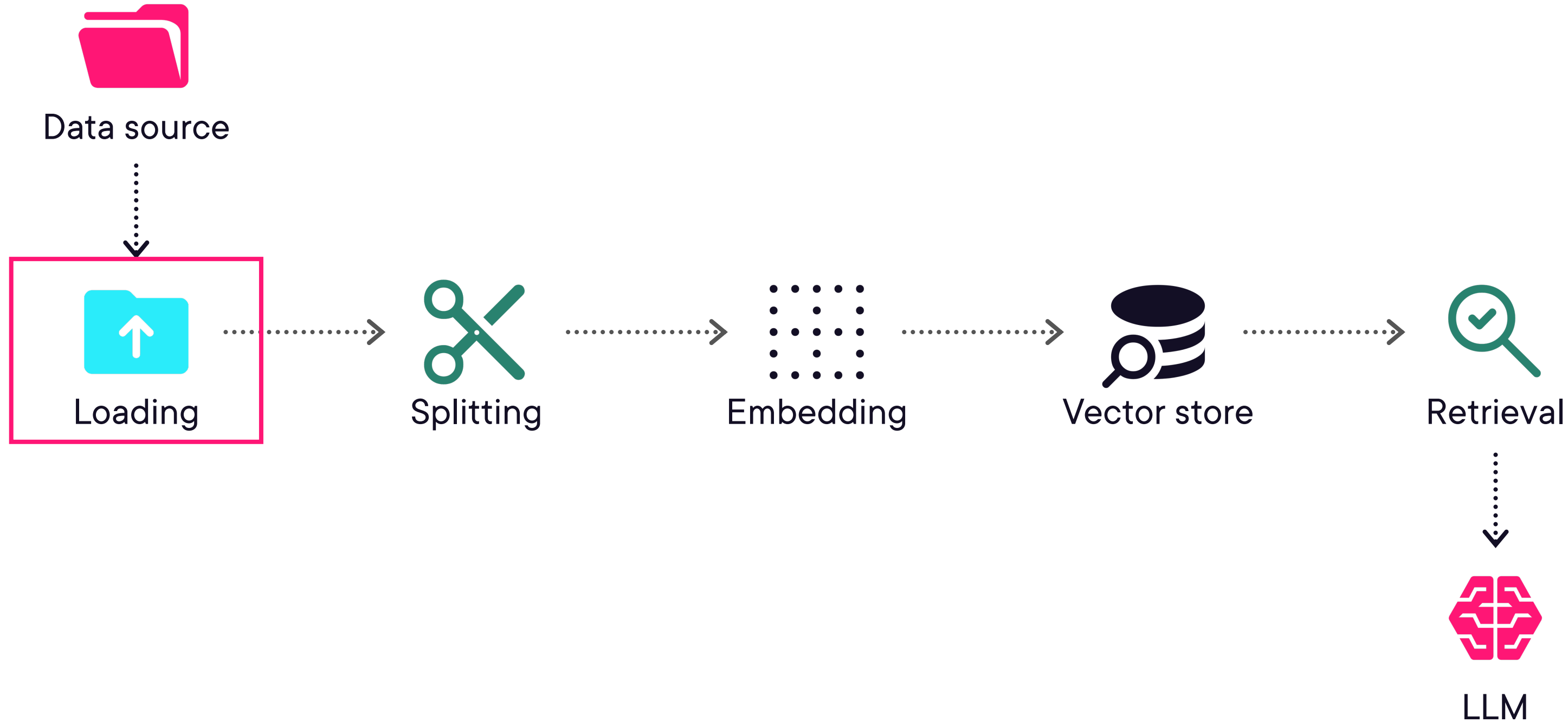
Data ingestion: First step in the RAG pipeline



Data ingestion: First step in the RAG pipeline



Data ingestion: First step in the RAG pipeline



Document loaders in LangChain

Convert data into a standardized format

Data sources are structured differently

- A PDF has pages and layouts
- A CSV has rows and columns
- A website contains HTML and scripts

Loaders abstract away format-specific complexity

- **PDF loader:** extracts text page-by-page
- **CSV loader:** interprets rows and columns
- **Web loader:** strips HTML and extracts readable text



Unified output from document loaders

All loaders produce
standardized
Document objects

Downstream
components work
independently of the
data source

Enables modular,
flexible, and
interchangeable
pipelines



Classification of document loaders



File-based loaders

Load data from local files like PDFs, CSVs, and documents



Web-based loaders

Extract readable content from web pages



API-based loaders

Fetch data directly from external APIs and online services



Database and cloud loaders

Load data from databases and cloud storage systems

Understanding the Document object



What is a Document **object**?



page_content

Contains the actual text extracted from the source that is ultimately sent to the LLM



metadata

Dictionary that stores contextual information about the main text

Common metadata examples



PDF loader

```
{  
'author': 'Jack',  
'source':  
'xy.pdf',  
'total_pages': 6,  
'page_label': '1'  
}
```



Web loader

```
{  
'source':  
'http.....',  
'title': '.....',  
'description': ' ...'  
,  
'language': 'en'  
}
```



CSV Loader

```
{  
'source':  
'rs.csv',  
'row' : 1  
}
```



Custom

```
{  
'dept': 'tech',  
'year': 2025,  
'type': 'policy'  
}
```

Why is metadata important?



Controlled retrieval

Enables precise filtering and improves retrieval accuracy



Source attribution

Shows where the answer came from, increasing user trust



Debugging

Helps trace answers back to their original source when something goes wrong



Working with common file-based document loaders



Common file-based loaders



TextLoader

- 1 file = 1 Document object
- File-level granularity
- Logs, system output files
- Exported text content



CSVLoader

- 1 row = 1 Document object
- Row-level granularity
- Product, inventory datasets
- Sales, analytics records



PyPDFLoader

- 1 page = 1 Document object
- Page-level granularity
- Academic documents
- Reports, manuals

How to use a document loader

Instantiate the loader with
required parameters

Call `.load()` to get a list of
Document objects



Demo

Loading a text file into a Document object

Text encoding in file loading



Encoding errors

Incorrect encoding can cause
`UnicodeDecodeError` or unreadable
characters



Best practice

Always specify encoding (commonly
`utf-8`) and update it if an error occurs

Demo

Loading CSV and PDF Files

Loading data from web and API sources

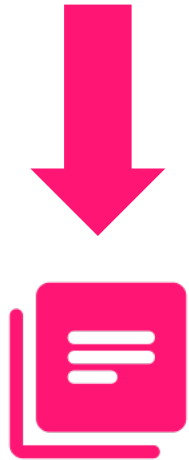


Loading data from the web



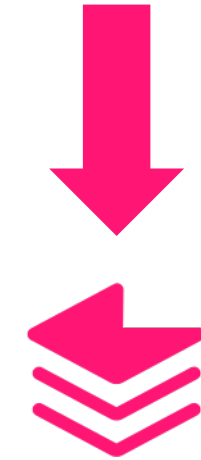
WebBaseLoader

Loads and extracts text from HTML web pages



WikipediaLoader

Retrieves structured article content using the Wikipedia API



ArxivLoader

Fetches research paper abstracts from arXiv API



Demo

Web and API-based loaders

Batch loading and processing large document collections

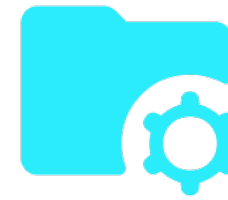


Scaling document ingestion



Manual loading

Requires writing extra logic for filtering, error handling, and managing directories



Scalable batch loading

Automatically handles directories, errors, and scaling with minimal code using `DirectoryLoader`



Understanding DirectoryLoader syntax and parameters

```
from langchain.document_loaders
import DirectoryLoader,
TextLoader

loader = DirectoryLoader(
    path="data/text_files",
    glob="**/*.txt",
    loader_cls=TextLoader
)
```

path tells where to look for documents

glob helps in filtering files

Examples:

- "*.txt" : text files from the current directory
- "*.pdf" : PDFs from the current directory
- "**/*.txt" : Loads from nested directories

loader_cls specifies which loader reads the files



Understanding DirectoryLoader syntax and parameters

```
from langchain.document_loaders
import DirectoryLoader,
TextLoader

loader = DirectoryLoader(
    path="data/text_files",
    glob="**/*.txt",
    loader_cls=TextLoader,
    loader_kwargs={"encoding":
                    "utf-8"},
    show_progress=True,
    silent_errors=True,
)
```

`loader_kwargs` passes additional configuration to the underlying loader

`show_progress` displays a progress bar during loading

`silent_errors` skips problematic files instead of stopping execution



Demo

Loading multiple text files

Demo

Loading multiple PDF files

Demo

Batch loading with progress and error handling

Lazy loading for memory-efficient document ingestion



Eager loading vs. lazy loading in LangChain

Eager loading

`loader.load()`

Creates and stores all document objects in memory at once and returns them as a complete list

Lazy loading

`loader.lazy_load()`

Generates and processes one document at a time, reducing memory usage and improving scalability



When and why to use lazy loading

Ideal for large directories, large PDFs, or memory-constrained systems

Most beneficial with `DirectoryLoader` handling many files



How it works: Eager vs. lazy loading

```
documents = loader.load()
```

```
for doc in documents:  
    process(doc)
```

Eager loading

Loads all documents into memory before processing

```
for doc in loader.lazy_load():  
    process(doc)
```

Lazy loading

Loads and processes one document at a time

Demo

Lazy loading with DirectoryLoader